

CUDA Thrust

Parallel and Heterogeneous Computing

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory

Department of Electrical and Computer Engineering

Cullen College of Engineering

University of Houston

UNIVERSITY of **HOUSTON** | ECE

C++ Templates (Review)

- Templates provide a method for writing C++ code independent of data type
- Data type is specified when a variable is declared
 - must be known at compile time
 - different versions of a template variable are created for each data type
- Specify a template parameter using:
`template<typename T>`
or
`template<class T>`
- Writing code for generic types is known as *generic programming*

Template Functions

- A template function can be defined:

```
template<typename T>
T max(T v0, T v1){
    if(v0 >= v1) return v0;
    return v1;
}
```

- The template function can be specified directly in the code using <...>

```
float v = max<float>(v0, v1)
```

- Or the compiler can infer the data type based on parameters

```
float v0 = 1.0f;
float v1 = 2.0f;
float v = max(v0, v1);
```

Common Template Problems

- You will generally get a compiler error if the template is inferred and ambiguous:

```
float v0 = 1.0f;  
double v1 = 2.0;  
float v = max(v0, v1); //it isn't clear which data type to use
```
- Template declarations and definitions have to be in the same file
 - you can't have a separate *.h file to store function declarations
 - since the data type is unknown when the function is written, pre-compiled libraries can't be made
- Operators used in template functions must be defined for the data type being used:

```
std::string s0 = "Hello ";  
std::string s1 = "world!";  
std::string s = max(s0, s1);           //what does this mean?
```

Template Class Example – Complex Numbers

- A complex number is defined as:

$$c = a + bi \quad \text{where } i^2 = -1$$

- An application may require various precisions for a and b
 - You can write multiple versions of a data structure to store a complex number

```
struct complex{  
    float a;  
    float b;  
}
```

```
struct complexDouble{  
    double a;  
    double b;  
}
```

- OR you can implement complex as a template class:

```
template<typename T>  
struct complex{  
    T a;  
    T b;  
}
```

Complex Numbers – Code

```
template<typename T>
class complex{
    T a;           //real part
    T b;           //imaginary part
public:
    //constructor
    complex(T real, T imag){
        a = real; b = imag;
    }
    //implement addition
    complex<T> operator+(complex<T> rhs){
        return complex(a + rhs.a,
                        b + rhs.b);
    }
}
```

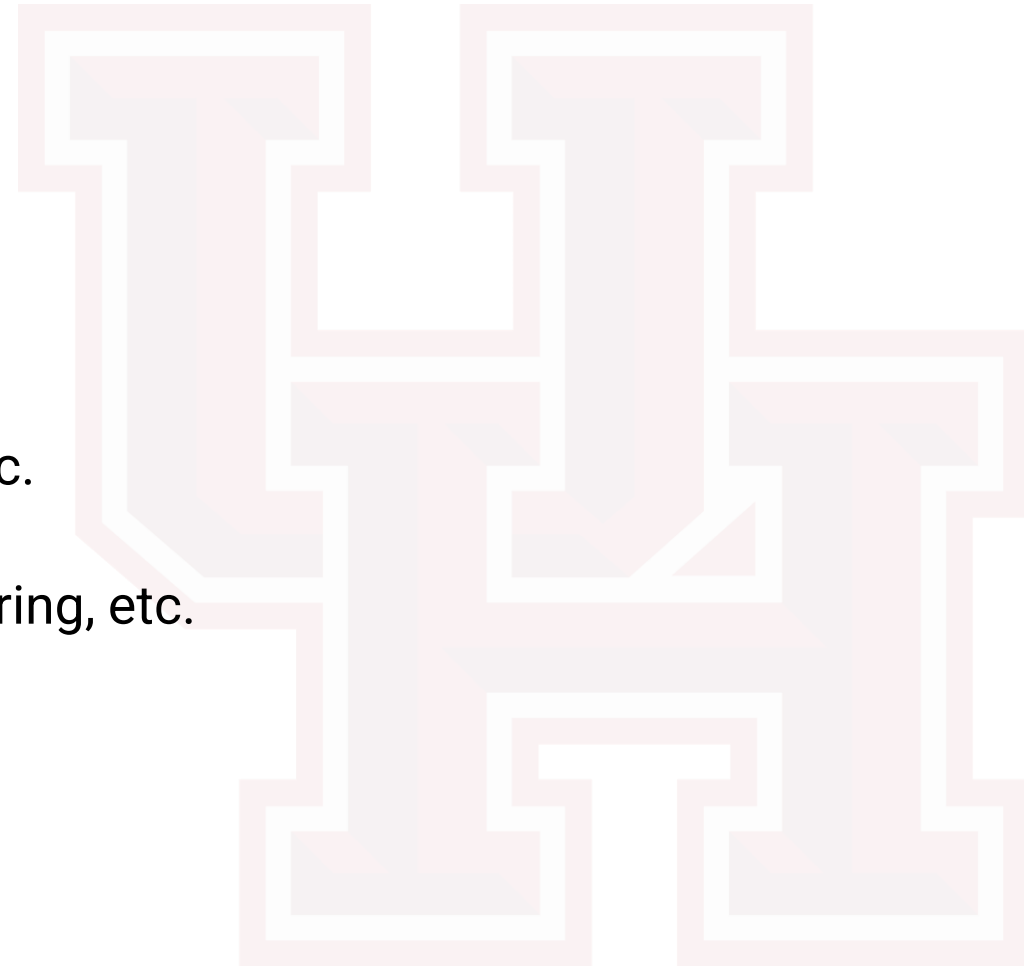
```
T operator-(...){...} //implement sub
T operator*(complex<T> rhs){
    return complex(a * rhs.a - b * rhs.b,
                  a * rhs.b + b * rhs.a);
}
T operator/(...){...} //implement div
} //end complex class
```

C++ Standard Template Library

- Standard Template Library (STL)
 - series of template-based algorithms and containers
 - developed primarily by Alexander Stepanov and Meng Lee
 - implementations are often distributed with compilers
 - Dinkum STL
 - Microsoft STL – distributed with Visual C++, based on Dinkum
 - libstdc++ – distributed with g++
- C++ Standard Library
 - based on conventions in the STL (several shared features)
 - part of the C++ ISO standard
 - currently C++14, with C++2017 under development
 - syntax and semantics are standardized
 - there are some performance standards
 - implementation is up to the compiler developer (Microsoft, Intel, etc.)

C++ Standard Template Library

- Implemented as a series of header files
- Includes the original ISO C90 C header files
 - `stdlib.h`, `math.h`, `time.h`, `complex.h`, etc.
- Template classes are in the `std` namespace
 - Containers
 - `std::list`, `std::vector`, `std::queue`, `std::array` (C++11), etc.
 - Input/Output and Strings
 - `std::cin`, `std::cout`, `std::ifstream`, `std::ofstream`, `std::string`, etc.
 - Numerical
 - `std::complex`, `std::valarray`, `std::accumulate(...)`, etc.
 - Algorithms
 - `std::sort(...)`, `std::find(...)`, `std::shuffle(...)`, etc.
 - Threads (C++11)



C++ STD Example – Containers

- A C++ vector (`std::vector`) is a variable-length template array

```
#include<vector>
```

```
...
```

```
                                //set or load values
```

```
std::vector<float> X(100, 0.0f); //array of 100 values initialized to 0
```

```
float v0 = X[20];
```

```
float v1 = X[19];
```

```
X.push_back(v0 + v1);
```

C++ STD Example – Nested Templates

- Templates can be nested

```
#include<vector>
```

```
#include<complex>
```

```
...
```

```
std::vector< complex<float> > X(100, 0.0f);
```

```
complex<float> c0 = X[20];
```

```
complex<float> c1 = X[21];
```

```
std::cout<<c0 + c1<<std::endl;
```

```
std::cout<<"real part: "<<(c0 + c1).real()<<std::endl;
```

```
std::cout<<"imag part: "<<(c0 + c1).imag()<<std::endl;
```

Thrust – nVidia's CUDA Template Library

- Vectors have host and device versions indicating storage location:

```
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
...

size_t N = 10;
thrust::host_vector<float> H(N);           //create a vector of N elements
H[0] = 1;                                  //set the first two elements to 1
H[1] = 1;
for(size_t n = 2; n < N; n++)              //for all of the other elements
    H[n] = H[n-1] + H[n-2];                 //calculate the Fibonacci sequence
thrust::device_vector<float> D = H;        //copy the result to the device
for(size_t n = 0; n < N; n++)
    std::cout<<"D["<<n<<" ] = "<<D[n]<<std::endl; //output device values
```

Thrust Compatibility with C++ STD

- Operator overloads support transforming `std::vector` objects to `thrust::host_vector` and `thrust::device_vector`

```
#include <vector>
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
...

size_t N = 10;
std::vector<float> V(N, 0.0f);           //create an array of 0's
thrust::host_vector<float> H = V;        //host_vector with values in V
thrust::device_vector<float> D = V;      //device_vector with values in V
for(size_t n = 0; n < N; n++)
    std::cout<<"D["<<n<<" ] = "<<D[n]<<std::endl; //output device values
```

Thrust Data Access

- What can we infer about accessing elements in `thrust::device_vector`?

```
thrust::device_vector<float> D;  
...  
for(size_t n = 0; n < N; n++)  
    std::cout<<"D["<<n<<" ] = "<<D[n]<<std::endl;
```

- accessing individual elements on the device requires a `cudaMemcpy(...)` call
- use these calls sparingly (ex. debugging)

- The functions `begin()` and `end()` return pointers to the beginning and end of an array

```
thrust::device_vector<float> D(13);  
thrust::sequence(D.begin(), D.end());
```

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]
0.0	1.0	2.0	3.0	4.0	5.0	6.0	7.0	8.0	9.0	10.0	11.0	12.0

```
thrust::fill(D.begin() + 2, D.begin() + 10, 99.0f);
```

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]
0.0	1.0	99.0	99.0	99.0	99.0	99.0	99.0	99.0	99.0	10.0	11.0	12.0

Thrust Transformations

- Algorithms that operate on every element are called *transformations*
- A *transform functor* is provided to define the transformation
- Calculate $y[i] = x[i] + c$ where $i \in [0, N - 1)$

```
#include <thrust/transform.h>
#include <thrust/functional.h>
std::vector<float> x(N);
float c;
...
thrust::device_vector<float> gpu_x = x;
thrust::device_vector<float> gpu_c(N);
thrust::fill(gpu_c.begin(), gpu_c.end(), c);
thrust::device_vector<float> y(N);
thrust::transform(gpu_x.begin(), gpu_x.end(),
                  gpu_c.begin(), gpu_y.begin(), thrust::plus<float>());
```

Thrust Transformations

- Calculate $g[n] = (f[n])^2$ where $n \in [0, N)$ and $f(x)$ is the x th Fibonacci number
- Define a function for evaluating the Fibonacci sequence

```
template <typename T>
thrust::host_vector<T> fib(N){
    thrust::host_vector<T> f(N);
    f[0] = f[1] = 1;
    for(size_t x = 2; x < N; x++)
        f[x] = f[x-1] + f[x-2];
    return f;
}
```

Thrust Transformations

- Calculate $g[n] = (f[n] + x[n])^2$ where $n \in [0, N)$, $x(n) = n$, and $f(n)$ is the n th Fibonacci number

```
size_t N;
```

```
...
```

```
thrust::device_vector<float> f = fib<float>(N);
```

```
thrust::device_vector<float> x(N);
```

```
thrust::sequence(x.begin(), x.end());           // x(n) = n
```

```
thrust::device_vector<float> y(N);
```

```
thrust::transform(f.begin(), f.end(), x.begin(),  
                 y.begin(), thrust::plus<float>());
```

```
thrust::device_vector<float> g(N);
```

```
thrust::transform(y.begin(), y.end(), y.begin(),  
                 g.begin(), thrust::multiplies<float>());
```

Defining Functors – saxpy()

- Calculate $y[i] = \alpha x[i] + y[i]$ where $i \in [1, N]$
 - this is the standard BLAS saxpy() function

```
void slow_saxpy(float alpha, thrust::device_vector<float>& x,  
               thrust::device_vector<float>& y){  
    thrust::device_vector<float> a(x.size()); //create a temp vector  
    thrust::fill(a.begin(), a.end(), alpha); //fill it with  $\alpha$   
    thrust::transform(x.begin(), x.end(), a.begin(), // $\alpha[i] = \alpha[i]*x[i]$   
                     a.begin(), thrust::multiplies<float>());  
    thrust::transform(a.begin(), a.end(), y.begin(), // $y[i] = y[i] + \alpha[i]$   
                     y.begin(), thrust::plus<float>());  
}
```

Defining Functors – saxpy()

- Use `slow_saxpy(...)` to calculate

$$y[i] = \alpha x[i] + y[i]$$

```
size_t N;           //stores number of elements
thrust::host_vector<float> X;  //stores the element array on the host
float a;            //stores the value for  $\alpha$ 
...                //load everything
thrust::device_vector<float> x = X;  //copy the array to the device
//create an output vector and initialize it to 0
thrust::device_vector<float> y(N, 0.0f);
slow_saxpy(a, x, y);  //perform the multiplication
```

Creating a Functor

- Create a functor structure that will be used in the transform

```
template <typename T>
struct saxpy_functor{
    const T a;
    saxpy_functor(T alpha) : a(alpha) {}; //constructor assigns  $\alpha$ 

    //create a function that will be applied to each element of a vector
    __host__ __device__ //usable on either platform
    T operator()(const T& x, const T& y) const{
        return a * x + y;
    }
}
```

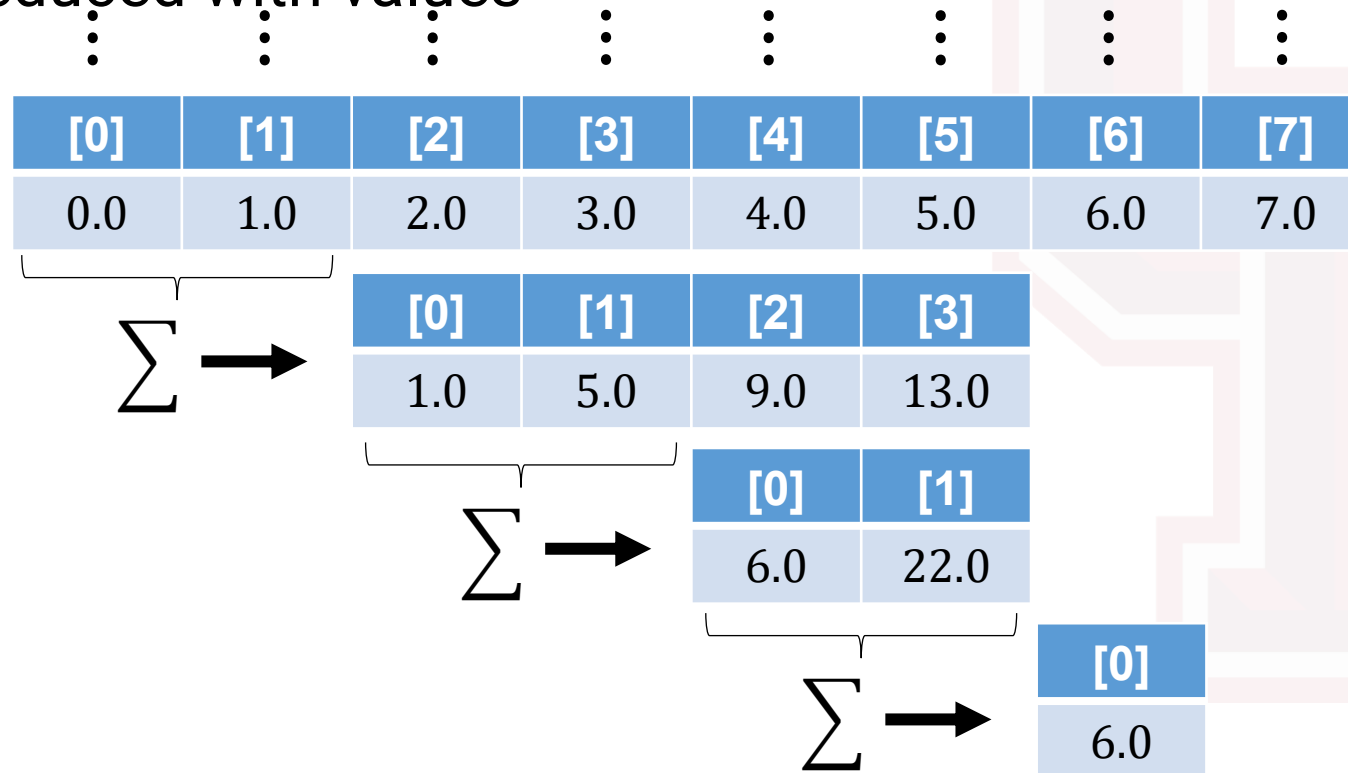
Using the saxpy(...) Functor

```
void slow_saxpy(float alpha, thrust::device_vector<float>& x,
               thrust::device_vector<float>& y){
    thrust::device_vector<float> a(x.size());           //create a temp vector
    thrust::fill(a.begin(), a.end(), alpha); //fill it with  $\alpha$ 
    thrust::transform(x.begin(), x.end(), a.begin(), // $\alpha[i] = \alpha[i]*x[i]$ 
                      a.begin(), thrust::multiplies<float>());
    thrust::transform(a.begin(), a.end(), y.begin(), // $y[i] = y[i] + \alpha[i]$ 
                      y.begin(), thrust::plus<float>());
}

void fast_saxpy(float alpha, thrust::device_vector<float>& x,
               thrust::device_vector<float>& y){
    thrust::transform(x.begin(), x.end(), y.begin(),
                      y.begin(), saxpy_functor(alpha));
}
```

Reductions

- A parallel reduction produces a small output from a large input
- Iteratively divide the data into pairs and reduce
 - occupancy is reduced with values



Multiplication Reduction

- Reduction function

```
template <typename T>
```

```
thrust::reduce(InputIterator first, InputIterator last, T init, BinaryFunc op)
```

- Calculate $n!$ in parallel

```
size_t n;
```

```
//stores the input
```

```
...
```

```
thrust::device_vector<size_t> x(n);
```

```
thrust::sequence(x.begin(), x.end(), 1, 1); //start at 1, spacing = 1
```

```
//perform the reduction transform using the thrust::multiplies functor
```

```
//the initial value of the result is 1 (0 would return 0)
```

```
size_t nfac = thrust::reduce(x.begin(), x.end(), (size_t) 1,  
                             thrust::multiplies<size_t>());
```

Scanning

- A scan applies the reduction operation to all values preceding an index

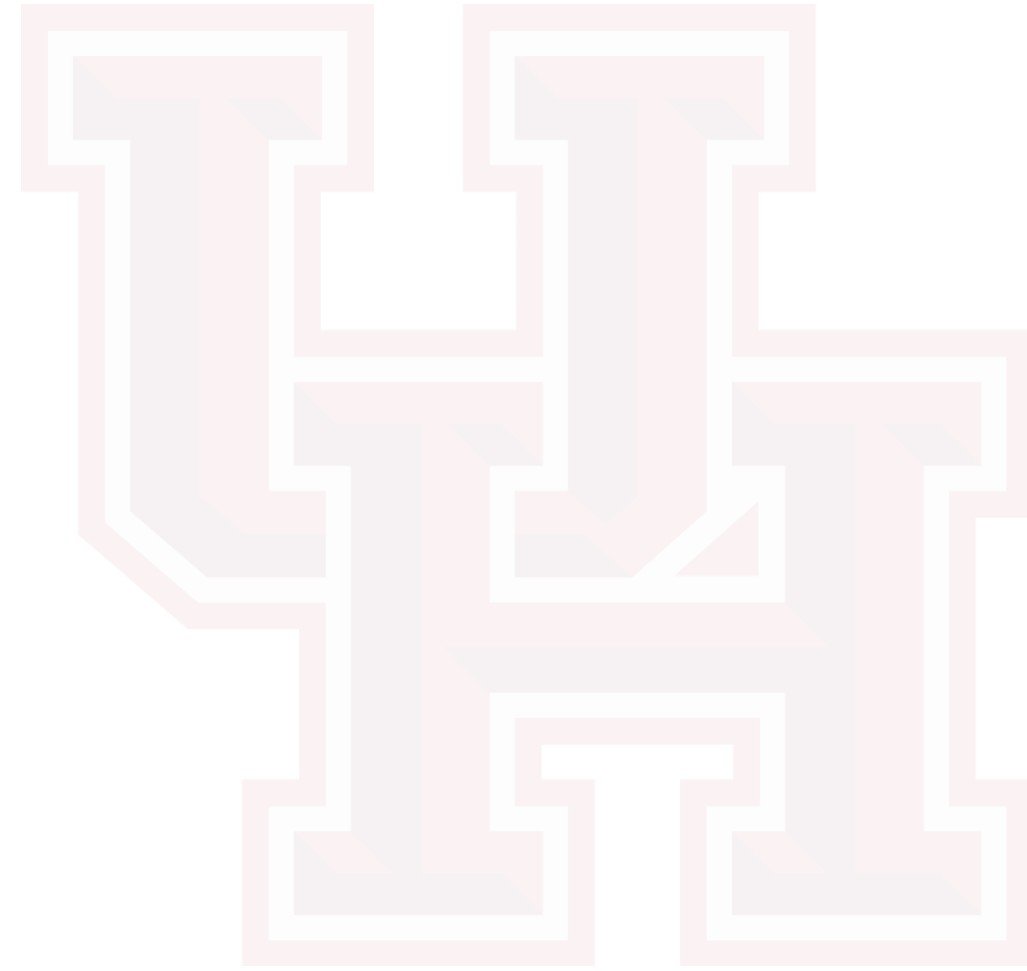
$$y[n] = \sum_{i=0}^{n-1} i$$

```
#include <thrust/scan.h>
size_t N = 100;
thrust::host_vector<float> x(N);
thrust::host_vector<float> y(N);
thrust::sequence(x.begin(), x.end());
//x = {0, 1, 2, 3, 4, 5, 6, ...}
thrust::inclusive_scan(x.begin(), x.end(), y.begin(), thrust::plus<float>());
//y = {0, 1, 3, 6, 10, 15, 21, ...} (includes the value at n)
thrust::exclusive_scan(x.begin(), x.end(), y.begin(), thrust::plus<float>());
//y = {0, 0, 1, 3, 6, 10, 15, ...} (excludes the value at n)
```

Sorting

- Sort an array

```
#include <thrust/sort.h>
...
size_t N;
thrust::device_vector<int> x;
... //fill x
thrust::sort(x.begin(), x.end());
```



Sort Using Keys

- Sort the array $x[i]$ using values in $y[i]$

```
const int N = 6;
thrust::host_vector<int> keys(N);
thrust::sequence(keys.begin(), keys.end(), (int)N, -1);
//keys = {6, 5, 4, 3, 2, 1}
thrust::host_vector<char> values(N);
thrust::sequence(values.begin(), values.end(), 'a', (char)1);
//values = {'a', 'b', 'c', 'd', 'e', 'f'}
thrust::sort_by_key(keys.begin(), keys.end(), values.begin());
//keys = {1, 2, 3, 4, 5, 6}
//values = {'f', 'e', 'd', 'c', 'b', 'a'}
```

Pairs

- A pair is a set of two values that can have different data types:

```
template <typename T1, typename T2>  
struct thrust::pair< T1, T2 >
```

```
#include <thrust/pair.h>
```

```
...
```

```
thrust::pair<char, double> a('a', 1.333); //create a char/double pair
```

```
thrust::pair<float, int> b(3.14159f, -2); //create a float/int pair
```

```
double f = a.second(); //f = 1.333
```

```
int i = b.first(); //error, loss of precision (float->int)
```

Tuples

- A tuple is an extension of the concept of a pair to additional types

```
#include <thrust/tuple.h>
...
size_t N = 3;
thrust::host_vector< thrust::tuple<int, const char*, float> > P(3);
P[0] = thrust::tuple<int, const char*, float>(1, "alice", 13.2f);
P[1] = thrust::tuple<int, const char*, float>(2, "feng", 16.7f);
P[2] = thrust::tuple<int, const char*, float>(3, "mohamed", 34.8f);

for(size_t i = 0; i < N; i++)
    std::cout<<thrust::get<0>(P[i])
              <<"      "<<thrust::get<1>(P[i])
              <<"      age: "<<thrust::get<2>(P[i])<<std::endl;
```